

REGISTRO DE MOBILE - SEMANA 15

RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DE ANIMAÇÕES AVANÇADAS

Componente Curricular: Desenvolvimento de Aplicativos Móveis

Tema: Animações Baseadas em Física e Transições Personalizadas

Contexto: Aplicativo de Receitas Culinárias

1. INTRODUÇÃO

Este relatório apresenta a solução para o problema de usabilidade detectado em um aplicativo de receitas culinárias. Os usuários relataram que a navegação entre as seções do ecossistema parecia "seca e abrupta". Para mitigar o problema e melhorar a experiência do usuário (UX), refinamos a interface utilizando animações baseadas na física e transições personalizadas de elementos compartilhados.

2. FERRAMENTA E AMBIENTE DE EXECUÇÃO

Para solucionar o problema, foi escolhido o ecossistema **Flutter**, utilizando a linguagem **Dart**.

Por que o Flutter?

- **Motor Gráfico Impeller/Skia:** Permite renderização a 60fps ou 120fps, garantindo que as animações físicas rodem sem travamentos.
- **Nativo Espacial:** Possui suporte nativo a simulações de molas (`SpringSimulation`) e transições fluidas de elementos (`Hero` widgets).

Como rodar este projeto:

1. Instale o **Flutter SDK** em sua máquina.
2. Configure uma IDE (como **VS Code** ou **Android Studio**).
3. Crie um projeto novo executando no terminal: `flutter create app_receitas`
4. Substitua o conteúdo do arquivo `lib/main.dart` pelo código fornecido na seção 4 deste relatório.
5. Execute em um emulador ou dispositivo físico usando o comando: `flutter run`

3. RESOLUÇÃO DA SITUAÇÃO-PROBLEMA (ROTEIRO)

3.1. Animações Baseadas na Física

- **Efeito Mola na Lista (Card Expandível):** Em vez de usar durações de tempo fixas e artificiais, aplicamos uma física de mola real. Ao tocar para expandir os detalhes rápidos de uma receita na lista, o cartão calcula o movimento baseado em massa e rigidez. O elemento ultrapassa suavemente o tamanho final e oscila antes de parar.
- **Efeito Rebote no Botão de Favoritos:** Ao clicar no ícone de coração, o sistema inicia um gatilho de escala cíclica rápida (contração para `0.8` e expansão instantânea para `1.3`, estabilizando em `1.0`), simulando a resistência mecânica de um botão real.

3.2. Transições Personalizadas

- **Elemento Compartilhado (Imagem da Receita):** Utilizamos o conceito de *Hero Animation*. A foto do prato na lista inicial expande-se de forma contínua até se transformar no topo (banner) da tela de detalhes, mantendo o foco visual estável.
- **Deslocamento de Texto (Slide-In):** O texto descritivo e o modo de preparo entram na tela deslizando da lateral com um leve atraso (*staggered animation*).
- **Retorno com Fade-out:** Ao clicar em voltar, a tela de detalhes reduz sua opacidade gradualmente até sumir por completo, revelando a lista por baixo de forma suave.

CÓDIGO PRINCIPAL NO VSCODE:

```
import 'package:flutter/material.dart';

import 'package:flutter/physics.dart';

void main() {

  runApp(const AppReceitas());

}

class AppReceitas extends StatelessWidget {

  const AppReceitas({super.key});
```

```
@override

Widget build(BuildContext context) {

  return MaterialApp(

    title: 'App de Receitas Avançado',

    debugShowCheckedModeBanner: false,

    theme: ThemeData(

      primarySwatch: Colors.orange,

      useMaterial3: true,

    ),

    home: const ListaReceitasPage(),

  );

}

}

// -----

// PÁGINA 1: LISTA DE RECEITAS (HOME)

// -----

class ListaReceitasPage extends StatelessWidget {

  const ListaReceitasPage({super.key});
```

```
@override

Widget build(BuildContext context) {

  return Scaffold(

    appBar: AppBar(

      title: const Text('Receitas Culinárias', style:
TextStyle(fontWeight: FontWeight.bold)),

      backgroundColor: Colors.orange.shade100,

    ),

    body: ListView(

      padding: const EdgeInsets.all(16),

      children: const [

        RecipeCardComponent(

          id: '1',

          titulo: 'Bolo de Chocolate Belga',

          calorias: '320 kcal',

          tempo: '40 min',

          imageUrl: 'https://unsplash.com',

        ),

        SizedBox(height: 16),
```

```

RecipeCardComponent(

    id: '2',

    titulo: 'Lasanha de Manjericão',

    calorias: '540 kcal',

    tempo: '60 min',

    imageUrl: 'https://unsplash.com',

),

],

),

);

}

}

// COMPONENTE: CARD DA RECEITA COM ANIMAÇÃO DE FÍSICA (MOLA)

class RecipeCardComponent extends StatefulWidget {

    final String id;

    final String titulo;

    final String calorias;

    final String tempo;

```

```
final String imageUrl;

const RecipeCardComponent({

  super.key,

  required this.id,

  required this.titulo,

  required this.calorias,

  required this.tempo,

  required this.imageUrl,

});

@override

State<RecipeCardComponent> createState() =>
_RcipeCardComponentState();
}

class _RecipeCardComponentState extends State<RecipeCardComponent> with
SingleTickerProviderStateMixin {

  late AnimationController _controller;

  double _scale = 1.0;
```

```
@override

void initState() {

    super.initState();

    _controller = AnimationController(

        vsync: this,

        upperBound: 0.05, // Define o limite máximo da oscilação física

    )..addListener(() {

        setState(() {});

    });

}

@override

void dispose() {

    _controller.dispose();

    super.dispose();

}

// Executa a simulação física de mola ao pressionar o Card
```

```
void _simularMola() {

    _controller.forward().then((_) {

        // Cria a física de mola retornando à posição inicial de
        equilíbrio

        final simulation = SpringSimulation(

            const SpringDescription(mass: 1.0, stiffness: 180.0, damping:
12.0),

            0.05, // Ponto de partida inicial

            0.0,  // Destino final estável

            0.0,  // Velocidade inicial

        );

        _controller.animateWith(simulation);

    });

}

@override

Widget build(BuildContext context) {

    _scale = 1.0 - _controller.value;

    return GestureDetector(
```



```
onTapDown: (_) => _simularMola(),

onTap: () {

    // Rota personalizada utilizando efeito contínuo Hero e
    Fade-out no retorno

    Navigator.push(

        context,

        MaterialPageRoute(

            transitionDuration: const Duration(milliseconds: 500),

            reverseTransitionDuration: const Duration(milliseconds:
300), // Fade-out ao voltar

            pageBuilder: (context, animation, secondaryAnimation) =>
DetalhesPage(

                id: widget.id,

                titulo: widget.titulo,

                imageUrl: widget.imageUrl,

            ),

            transitionsBuilder: (context, animation,
secondaryAnimation, child) {

                // Aplica Fade-out suave na saída da tela

                return FadeTransition(opacity: animation, child: child);

            },

        ),

    ),

}
```

```

    ),

    );

  },

  child: Transform.scale(

    scale: _scale,

    child: Card(

      elevation: 4,

      shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(16)),

      child: Column(

        crossAxisAlignment: CrossAxisAlignment.start,

        children: [

          // Elemento Compartilhado (Hero) aplicado à Imagem

          Hero(

            tag: 'image_${widget.id}',

            child: ClipRRect(

              borderRadius: const BorderRadius.vertical(top:
Radius.circular(16)),

              child: Image.network(widget.imageUrl, height: 180,
width: double.infinity, fit: BoxFit.cover),

            ),

```

```

    ),

    Padding(

      padding: const EdgeInsets.all(16),

      child: Row(

        mainAxisAlignment: MainAxisAlignment.between,

        children: [

          Column(

            crossAxisAlignment: CrossAxisAlignment.start,

            children: [

              Text(widget.titulo, style: const
TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),

              const SizedBox(height: 4),

              Text('${widget.tempo} • ${widget.calorias}',
style: const TextStyle(color: Colors.grey)),

            ],

          ),

          BotaoFavoritoFisico(), // Botão com efeito rebote
integrado

        ],

      ),

    )

```

```

        ],

        ),

    ),

    ),

);

}

}

// COMPONENTE: BOTÃO FAVORITOS COM EFEITO DE REBOTE (BOUNCE)

class BotaoFavoritoFisico extends StatefulWidget {

    const BotaoFavoritoFisico({super.key});

    @override

    State<BotaoFavoritoFisico> createState() =>
    _BotaoFavoritoFisicoState();

}

class _BotaoFavoritoFisicoState extends State<BotaoFavoritoFisico> with
SingleTickerProviderStateMixin {

    late AnimationController _controller;

```

```
bool _isFavorited = false;

@override

void initState() {

    super.initState();

    _controller = AnimationController(

        duration: const Duration(milliseconds: 150),

        vsync: this,

    );

}

@override

void dispose() {

    _controller.dispose();

    super.dispose();

}

void _dispararRebote() {

    setState(() {
```

```

        _isFavorited = !_isFavorited;

    });

    // Executa sequência dinâmica de encolhimento e expansão repentina
    (Rebote)

    _controller.forward().then((_) => _controller.reverse());

}

@override

Widget build(BuildContext context) {

    return ScaleTransition(

        scale: Tween<double>(begin: 1.0, end: 1.4).animate(

            CurvedAnimation(parent: _controller, curve: Curves.bounceOut),

        ),

        child: IconButton(

            icon: Icon(

                _isFavorited ? Icons.favorite : Icons.favorite_border,

                color: _isFavorited ? Colors.red : Colors.grey,

                size: 28,

            ),

            onPressed: _dispararRebote,

```

```
        ),

    );

}

}

// -----

// PÁGINA 2: DETALHES DA RECEITA (TRANSIÇÃO DE ENTRADA)

// -----

class DetalhesPage extends StatelessWidget {

    final String id;

    final String titulo;

    final String imageUrl;

    const DetalhesPage({

        super.key,

        required this.id,

        required this.titulo,

        required this.imageUrl,

    });
```

```
@override

Widget build(BuildContext context) {

  return Scaffold(

    body: SingleChildScrollView(

      child: Column(

        crossAxisAlignment: CrossAxisAlignment.start,

        children: [

          Stack(

            children: [

              // Recebe o elemento compartilhado da tela anterior de
forma fluida

              Hero(

                tag: 'image_$id',

                child: Image.network(imageUrl, height: 300, width:
double.infinity, fit: BoxFit.cover),

              ),

              Positioned(

                top: 40,

                left: 16,
```



```

        child: CircleAvatar(

          backgroundColor: Colors.white,

          child: IconButton(

            icon: const Icon(Icons.arrow_back, color:
Colors.black),

            onPressed: () => Navigator.pop(context),

          ),

        ),

      ),

    ],

  ),

  Padding(

    padding: const EdgeInsets.all(24),

    child: Column(

      crossAxisAlignment: CrossAxisAlignment.start,

      children: [

        // Texto deslizando da lateral (Slide-In) usando
TweenAnimation

        TweenAnimationBuilder<Offset>(

          duration: const Duration(milliseconds: 600),

```

```
        curve: Curves.easeOutCubic,

        tween: Tween<Offset>(begin: const Offset(100, 0),
end: const Offset(0, 0)),

        builder: (context, offset, child) {

            return Transform.translate(

                offset: offset,

                child: child,

            );

        },

        child: Text(

            titulo,

            style: const TextStyle(fontSize: 26, fontWeight:
FontWeight.bold, color: Colors.black87),

        ),

    ),

    const SizedBox(height: 16),

    const Text(

        "Modo de Preparo:",

        style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold, color: Colors.orange),

    ),
```

```
const SizedBox(height: 8),  
  
const Text(  
  
    "1. Misture os ingredientes com calma.\n"  
  
    "2. Adicione os elementos simulados da física de  
mola.\n"
```

Perguntas de Consolidação:

1. Qual é a importância das animações baseadas em Física para a experiência do usuário em um aplicativo móvel?

As animações baseadas em física aproximam a interface digital do mundo real, simulando propriedades como inércia, fricção, força e gravidade. Em vez de objetos acelerarem de forma linear e artificial, eles ganham "peso". Isso gera previsibilidade cognitiva: o cérebro humano, habituado com as leis naturais da física cotidiana, entende o movimento instantaneamente. O resultado direto é um feedback visual confortável, natural e de alta qualidade perceptiva.

2. Como as transições personalizadas podem impactar a percepção de um usuário ao navegar pelo aplicativo?

Transições personalizadas eliminam as mudanças abruptas de estado de tela (cortes secos, telas piscando ou surgimentos repentinos), substituindo-os por uma continuidade visual. No caso da técnica de elementos compartilhados (*Shared Elements*), o usuário consegue manter o foco espacial: ele vê exatamente para onde o componente original migrou e de onde o novo conteúdo surgiu. Isso diminui consideravelmente a carga de esforço mental necessária para se localizar dentro do fluxo estrutural do aplicativo.

3. Cite um exemplo de quando uma animação poderia ser excessiva e prejudicar a experiência do usuário.

Uma animação se torna prejudicial quando ela prioriza a vaidade estética em detrimento do desempenho ou da velocidade de execução de uma tarefa. ****Exemplo prático:**** Um menu de configurações em que cada opção secundária realiza uma animação complexa de rotação e desfoque que dura 1,5 segundo para abrir. O usuário que interage com o

aplicativo com frequência busca eficiência; movimentos muito lentos, excessivamente decorativos ou repetitivos em tarefas cotidianas geram atrito, irritação e passam a falsa impressão de lentidão no sistema.

4. Qual é o benefício de utilizar animações que simulam o comportamento de "mola" ao interagir com elementos da interface?

O principal benefício do efeito "mola" (*Spring*) é a entrega de uma resposta tátil altamente reativa aos comandos do usuário. Quando um elemento da interface responde elasticamente ao toque, ultrapassando um limite e oscilando sutilmente até retornar ao ponto de equilíbrio, ele simula a mecânica de botões reais do mundo físico. Essa maleabilidade faz com que a interface pareça viva, orgânica e responsiva, aumentando o índice de satisfação na manipulação e confirmação de ações no sistema.

CONCLUSÃO

A transição de uma interface estática para um modelo enriquecido com animações dinâmicas resolveu eficientemente a experiência áspera relatada no aplicativo de receitas. A combinação estratégica de forças físicas nas listagens e continuidades fluidas de tela transformou a usabilidade geral do produto, provando que boas animações não atuam como meros adornos visuais, mas sim como guias fundamentais da jornada do usuário.

Se precisar que eu faça qualquer ****ajuste no design do código**** ou que mude alguma das ****respostas explicativas****, conte comigo! Como deseja proceder com o envio do seu trabalho?